

Spring Security

Fundamentos, Criptografía y Arquitectura Interna

Facultad de Ingeniería — Computación en Red 2 — Semana 9

Autenticación vs. Autorización

Los Dos Pilares del Control de Acceso

Autenticación (¿Quién eres?)



Fase 1: Verificación de Identidad

Contrasta credenciales enviadas (usuario y contraseña) contra la persistencia.



Resultado Exitoso:

Construye el objeto `Authentication`.

Fallo:

Retorna

401 Unauthorized

Los Dos Pilares del Control de Acceso

Autenticación (¿Quién eres?)



Fase 1: Verificación de Identidad

Contrasta credenciales enviadas (usuario y contraseña) contra la persistencia.



Resultado Exitoso:

Construye el objeto `Authentication`.

Fallo:

Retorna

401 Unauthorized

.

Autorización (¿Qué puedes hacer?)



Fase 2: Concesión de Privilegios

Evalúa si la identidad posee los permisos requeridos para la ruta o método.



Resultado Exitoso:

Permite ejecución del controlador (200 OK).

Fallo:

Retorna

403 Forbidden

.

Flujo Conceptual: De la Identidad a los Privilegios

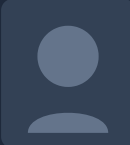
Autenticación vs. Autorización

Los dos pilares fundamentales de la seguridad a nivel de aplicación (Application-Level Security)

FASE 1: IDENTIDAD

Autenticación (Authentication)

Pregunta clave: ¿Quién eres? (Who are you?)



USER IDENTITY
Principal: `miUsuario`
Credentials: `.....` (Verificado)
AUTHENTICATED

MECANISMOS DE VALIDACIÓN

HTTP Basic

Form Login

JWT / OAuth2

Resultado: Se crea el objeto **Authentication** en memoria.

OK

FASE 2: PERMISOS

Autorización (Authorization)

Pregunta clave: ¿Qué puedes hacer? (What can you do?)

ROLES (WHO YOU ARE)

- `ROLE_ADMIN`
- `ROLE_MANAGER`
- `ROLE_CLIENT`

AUTHORITIES (CAN DO)

- `READ_PRODUCTS`
- `WRITE_ORDER`
- `DELETE_RECORD`

PUNTOS DE VERIFICACIÓN

Filtros HTTP (Web)

@PreAuthorize (Métodos)

Resultado: Permite (**200 OK**) o deniega (**403 Forbidden**).

Criptografía y Hashing de Contraseñas

Tres Paradigmas de Transformación de Datos

Codificación



Propósito:

Transporte de bytes en canales de texto (Base64, ASCII).



Cero Secreto:

100% reversible sin claves criptográficas.

Tres Paradigmas de Transformación de Datos

Codificación



Propósito:

Transporte de bytes en canales de texto (Base64, ASCII).



Cero Secreto:

100% reversible sin claves criptográficas.

Encriptación



Propósito:

Confidencialidad de información (AES-256, RSA).



Bidireccional:

Reversible solo por poseedores de la clave secreta.

Tres Paradigmas de Transformación de Datos

Codificación



Propósito:

Transporte de bytes en canales de texto (Base64, ASCII).



Cero Secreto:

100% reversible sin claves criptográficas.

Encriptación



Propósito:

Confidencialidad de información (AES-256, RSA).



Bidireccional:

Reversible solo por poseedores de la clave secreta.

Funciones Hash



Propósito:

Integridad y contraseñas seguras (BCrypt, Argon2).



Unidireccional:

Trampa matemática matemáticamente irreversible.

Comparativa: Codificación, Encriptación y Hashing

Codificación, Encriptación y Hashing

Comparativa de los 3 procesos de transformación de datos (Laurentiu Spilca — Spring Security)

FORMATO

1. Codificación (Encoding)

Cambio de formato para transporte / lectura.

Ej: Base64, ASCII, URL Encoding

Sin secretos: utiliza estándares públicos.

INPUT (ORIGEN)

"admin:123"

BASE64

OUTPUT (ENCODED)

"YWRTaW46MTIz"

100% REVERSIBLE

PROPIEDAD CLAVE

Cero Confidencialidad

No protege el secreto.

Cualquiera puede decodificarlo de manera inmediata sin claves.

CONFIDENCIALIDAD

2. Encriptación (Encryption)

Cifrado reversible protegido por una clave.

Ej: AES-256, RSA, TLS/HTTPS

Protege datos en tránsito y en reposo.

PLAINTEXT

"Tarjeta 4509"

+ CLAVE

CIPHERTEXT (HEX)

"8f4e2a1b9c..."

REQUIERE SECRET KEY

PROPIEDAD CLAVE

Bidireccional con Llave

Existe función inversa.

Solo quien posee la clave secreta puede recuperar el dato.

UNIDIRECCIONAL

3. Funciones Hash (Hashing)

Trampa matemática: irreversible por diseño.

Ej: BCrypt, Argon2, PBKDF2

Uso obligatorio para contraseñas.

PASSWORD PLANA

"miClave123"

+ Salt Aleatorio

HASH

DIGEST (60 CHAR)

"\$2a\$10\$wK...9xZ"

Longitud siempre fija

IRREVERSIBLE (-X-)

VERIFICACIÓN SEGURA

Nunca se descrypta

Se calcula el hash de la contraseña ingresada y se compara con matches().

La Amenaza: Rainbow Tables y Fuerza Bruta

Hashes Rápidos y Deterministas

Algoritmos como MD5 o SHA-256 fueron diseñados para checksums, no para contraseñas:



Tablas Rainbow:

Hashes precalculados que permiten revertir claves comunes al instante.



Aceleración por GPU:

Capacidad de procesar billones de hashes por segundo.

La Amenaza: Rainbow Tables y Fuerza Bruta

Hashes Rápidos y Deterministas

Algoritmos como MD5 o SHA-256 fueron diseñados para checksums, no para contraseñas:



Tablas Rainbow:

Hashes precalculados que permiten revertir claves comunes al instante.



Aceleración por GPU:

Capacidad de procesar billones de hashes por segundo.

Defensa Criptográfica con BCrypt

BCrypt neutraliza ambos ataques mediante dos técnicas matemáticas integradas:



Auto-Salting Aleatorio:

Inyecta 16 bytes únicos. Claves idénticas producen hashes distintos.



Work Factor Adaptativo:

Costo exponencial ($2^{12} = 4,096$ rondas) que ralentiza a atacantes.



60 Caracteres



Work Factor



Auto-Salting



Anti-Brute Force

Anatomía del Hash BCrypt

```
$2a$12$e8ukVsp3T0nqzGYZZ.24UeL7C8Z2g3aC9pS5vK7mQ2yW8x1z4v6u0
```



\$2a

—
Versión:

Algoritmo BCrypt estándar OpenBSD.



60 Caracteres



Work Factor



Auto-Salting



Anti-Brute Force

Anatomía del Hash BCrypt

```
$2a$12$e8ukVsp3T0nqzGYZZ.24UeL7C8Z2g3aC9pS5vK7mQ2yW8x1z4v6u0
```



\$2a

—
Versión:

Algoritmo BCrypt estándar OpenBSD.



\$12\$

—
Work Factor:

$2^{12} = 4,096$ rondas (~250ms por validación).



60 Caracteres



Work Factor



Auto-Salting



Anti-Brute Force

Anatomía del Hash BCrypt

```
$2a$12$e8ukVsp3T0nqzGYZZ.24UeL7C8Z2g3aC9pS5vK7mQ2yW8x1z4v6u0
```



\$2a

—
Versión:

Algoritmo BCrypt estándar OpenBSD.



\$12\$

—
Work Factor:

$2^{12} = 4,096$ rondas (~250ms por validación).



Salt (22 chars):

16 bytes aleatorios incrustados que invalidan Rainbow Tables.



60 Caracteres



Work Factor



Auto-Salting



Anti-Brute Force

Anatomía del Hash BCrypt

```
$2a$12$e8ukVsp3T0nqzGYZZ.24UeL7C8Z2g3aC9pS5vK7mQ2yW8x1z4v6u0
```



\$2a

Versión:

Algoritmo BCrypt estándar OpenBSD.



\$12\$

Work Factor:

$2^{12} = 4,096$ rondas (~250ms por validación).



Salt (22 chars):

16 bytes aleatorios incrustados que invalidan Rainbow Tables.



Digest (31 chars):

Salida criptográfica matemáticamente protegida.

Arquitectura de Spring Security

El Servlet Container y la Cadena de Filtros

El Puente: DelegatingFilterProxy

Tomcat no conoce el ApplicationContext de Spring. Utiliza un filtro estándar Jakarta:



DelegatingFilterProxy:

Captura la petición y delega al bean `springSecurityFilterChain`.



FilterChainProxy:

Gestor maestro que orquesta los filtros activos de seguridad.

El Servlet Container y la Cadena de Filtros

El Puente: DelegatingFilterProxy

Tomcat no conoce el ApplicationContext de Spring. Utiliza un filtro estándar Jakarta:



DelegatingFilterProxy:

Captura la petición y delega al bean `springSecurityFilterChain`.



FilterChainProxy:

Gestor maestro que orquesta los filtros activos de seguridad.

La Cadena: SecurityFilterChain

Filtros ordenados que inspeccionan la petición estrictamente en serie:



Seguridad Perimetral:

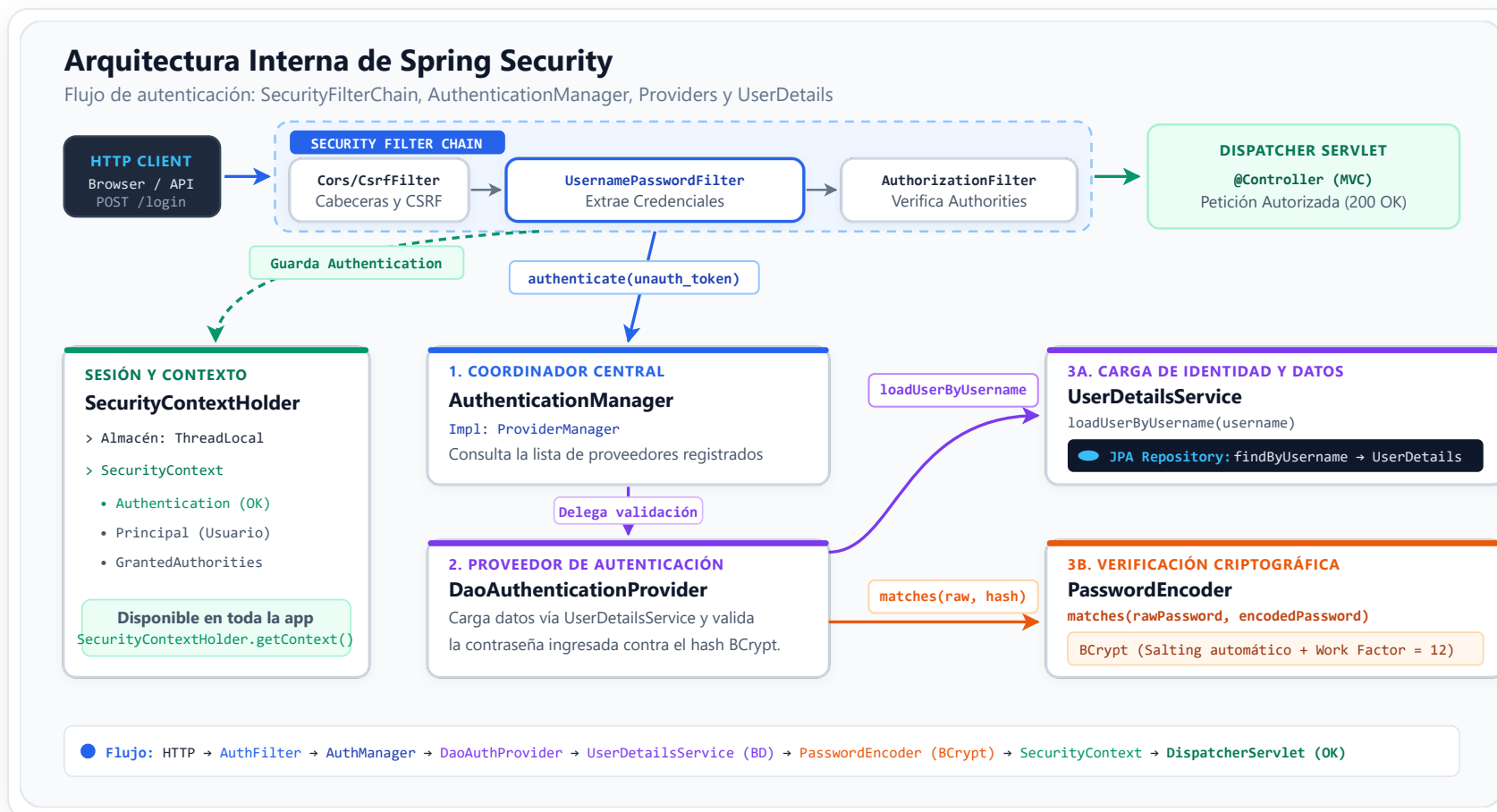
`CorsFilter` y `CsrfFilter` sanitizan la cabecera.



Autenticación:

Extrae credenciales antes de alcanzar el `DispatcherServlet`.

Arquitectura Interna: El Circuito Completo



El Flujo de Autenticación Paso a Paso

FILTRO DE LOGIN

`UsernamePasswordAuthenticationFilter`

Intercepta `POST /login`, extrae las credenciales y crea el token no autenticado.

El Flujo de Autenticación Paso a Paso

FILTRO DE LOGIN

`UsernamePasswordAuthenticationFilter`

Intercepta `POST /login`, extrae las credenciales y crea el token no autenticado.

AUTHENTICATIONMANAGER

`AuthenticationManager`
(`ProviderManager`)

Coordinador central que consulta la lista de proveedores hasta encontrar el adecuado.

El Flujo de Autenticación Paso a Paso

FILTRO DE LOGIN

`UsernamePasswordAuthenticationFilter`

Intercepta `POST /login`, extrae las credenciales y crea el token no autenticado.

AUTHENTICATIONMANAGER

`AuthenticationManager`
(`ProviderManager`)

Coordinador central que consulta la lista de proveedores hasta encontrar el adecuado.

DAOAUTHPROVIDER

`DaoAuthenticationProvider`

Carga datos con `UserDetailsService` y valida el hash con `PasswordEncoder`.

El Flujo de Autenticación Paso a Paso

FILTRO DE LOGIN

`UsernamePasswordAuthenticationFilter`

Intercepta `POST /login`, extrae las credenciales y crea el token no autenticado.

AUTHENTICATIONMANAGER

`AuthenticationManager`
(`ProviderManager`)

Coordinador central que consulta la lista de proveedores hasta encontrar el adecuado.

DAOAUTHPROVIDER

`DaoAuthenticationProvider`

Carga datos con `UserDetailsService` y valida el hash con `PasswordEncoder`.

SECURITYCONTEXT

`SecurityContextHolder`
(`ThreadLocal`)

Almacena en memoria el objeto `Authentication` validado durante toda la petición.

Los Contratos de Persistencia y Cifrado

Identidad: UserDetailsService

Interfaz funcional que conecta JPA con el ecosistema de seguridad:

```
UserDetails loadUserByUsername(String username)
```



UserDetails:

Abstracción con username, password hasheado y permisos (`getAuthorities()`).

Los Contratos de Persistencia y Cifrado

Identidad: UserDetailsService

Interfaz funcional que conecta JPA con el ecosistema de seguridad:

```
UserDetails loadUserByUsername(String username)
```



UserDetails:

Abstracción con username, password hasheado y permisos (`getAuthorities()`).

Criptografía: PasswordEncoder

Contrato estándar para validar credenciales:

```
boolean matches(CharSequence raw, String encoded)
```



Proceso:

Extrae el salt del hash, aplica BCrypt sobre la clave ingresada y compara resultados sin descifrar.



ThreadLocal



Principal



Authentication



Auto-Cleanup

El Contexto de Seguridad en Memoria

Una vez validado el usuario, Spring Security almacena la identidad en memoria para evitar re-autenticar en cada componente:



SecurityContextHolder:

Punto de acceso estático global para obtener el `SecurityContext` .



ThreadLocal



Principal



Authentication



Auto-Cleanup

El Contexto de Seguridad en Memoria

Una vez validado el usuario, Spring Security almacena la identidad en memoria para evitar re-autenticar en cada componente:



SecurityContextHolder:

Punto de acceso estático global para obtener el `SecurityContext`.



Estrategia ThreadLocal:

La información se asocia exclusivamente al hilo de ejecución del Servlet actual.



ThreadLocal



Principal



Authentication



Auto-Cleanup

El Contexto de Seguridad en Memoria

Una vez validado el usuario, Spring Security almacena la identidad en memoria para evitar re-autenticar en cada componente:



SecurityContextHolder:

Punto de acceso estático global para obtener el `SecurityContext`.



Estrategia ThreadLocal:

La información se asocia exclusivamente al hilo de ejecución del Servlet actual.



Ciclo de Vida:

Al finalizar la respuesta HTTP, el filtro ejecuta `SecurityContextHolder.clearContext()` evitando fugas de memoria.

Modelo de Autorización

Permisos Atómicos vs. Perfiles de Usuario

GrantedAuthority (Acciones)

Permisos atómicos y granulares sobre operaciones específicas:



Operaciones CRUD:

`READ_PRODUCT` , `CREATE_ORDER` , `DELETE_USER` .



Evaluación Literal:

`hasAuthority( 'READ_PRODUCT' )` verifica la cadena exacta sin prefijos.

Permisos Atómicos vs. Perfiles de Usuario

GrantedAuthority (Acciones)

Permisos atómicos y granulares sobre operaciones específicas:



Operaciones CRUD:

`READ_PRODUCT` , `CREATE_ORDER` , `DELETE_USER` .



Evaluación Literal:

`hasAuthority('READ_PRODUCT')` verifica la cadena exacta sin prefijos.

Roles (Agrupaciones)

Perfiles de alto nivel que representan cargos en la organización:



Perfiles:

`ROLE_ADMIN` , `ROLE_MANAGER` , `ROLE_CLIENT` .



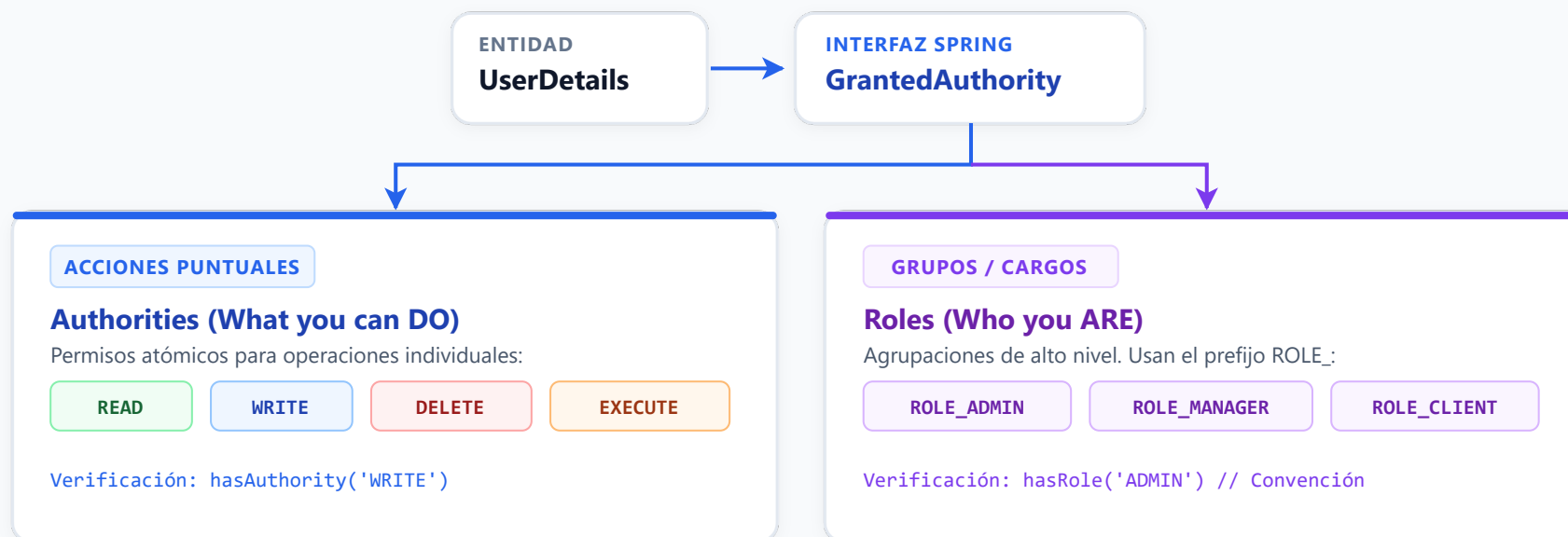
Convención Spring:

Un rol es técnicamente una `GrantedAuthority` que inicia con `ROLE_` .

GrantedAuthority: Permisos vs. Roles

GrantedAuthority: Authorities vs. Roles

El modelo de control de acceso en Spring Security (Laurentiu Spilca — Lesson 1)



- **Regla de Oro en Spring Security:** Un Role es técnicamente una GrantedAuthority que comienza con el prefijo `ROLE_`.
Authority: Acción atómica puntual (permiso). **Role:** Perfil o cargo organizativo del usuario.



Prefijo ROLE_



hasRole()



hasAuthority()



Desacoplamiento

La Regla de Oro del Prefijo ROLE_

La principal fuente de errores en seguridad proviene de mezclar roles y authorities en código:



Con hasRole('ADMIN'):

Spring antepone internamente `ROLE_` y busca la autoridad `ROLE_ADMIN`.



Prefijo ROLE_



hasRole()



hasAuthority()



Desacoplamiento

La Regla de Oro del Prefijo ROLE_

La principal fuente de errores en seguridad proviene de mezclar roles y authorities en código:



Con hasRole('ADMIN'):

Spring antepone internamente `ROLE_` y busca la autoridad `ROLE_ADMIN`.



Con hasAuthority('ROLE_ADMIN'):

Comprueba coincidencia literal exacta. Equivale exactamente a `hasRole('ADMIN')`.



Prefijo ROLE_



hasRole()



hasAuthority()



Desacoplamiento

La Regla de Oro del Prefijo ROLE_

La principal fuente de errores en seguridad proviene de mezclar roles y authorities en código:



Con hasRole('ADMIN'):

Spring antepone internamente `ROLE_` y busca la autoridad `ROLE_ADMIN`.



Con hasAuthority('ROLE_ADMIN'):

Comprueba coincidencia literal exacta. Equivale exactamente a `hasRole('ADMIN')`.



Buena Práctica:

Asegurar controladores con permisos atómicos (`hasAuthority`) y mapear roles en persistencia.



BCrypt Obligatorio



Authn vs Authz



Permisos Granulares



SecurityFilterChain

Resumen de Buenas Prácticas

Las 4 Reglas de Oro en Spring Security



1. Passwords Siempre con BCrypt:

Jamás guardar contraseñas en texto plano ni con MD5/SHA. Usar `BCryptPasswordEncoder` (work factor ≥ 10).



BCrypt Obligatorio



Authn vs Authz



Permisos Granulares



SecurityFilterChain

Resumen de Buenas Prácticas

Las 4 Reglas de Oro en Spring Security



1. Passwords Siempre con BCrypt:

Jamás guardar contraseñas en texto plano ni con MD5/SHA. Usar `BCryptPasswordEncoder` (work factor ≥ 10).



2. Distinción Estricta Authn vs Authz:

Autenticación produce 401; autorización produce 403. Mantener ambos flujos desacoplados.



BCrypt Obligatorio



Authn vs Authz



Permisos Granulares



SecurityFilterChain

Resumen de Buenas Prácticas

Las 4 Reglas de Oro en Spring Security



1. Passwords Siempre con BCrypt:

Jamás guardar contraseñas en texto plano ni con MD5/SHA. Usar `BCryptPasswordEncoder` (work factor ≥ 10).



2. Distinción Estricta Authn vs Authz:

Autenticación produce 401; autorización produce 403. Mantener ambos flujos desacoplados.



3. Autorización Granular:

Asegurar controladores con permisos atómicos y mapear roles a conjuntos de permisos en persistencia.



BCrypt Obligatorio



Authn vs Authz



Permisos Granulares



SecurityFilterChain

Resumen de Buenas Prácticas

Las 4 Reglas de Oro en Spring Security



1. Passwords Siempre con BCrypt:

Jamás guardar contraseñas en texto plano ni con MD5/SHA. Usar `BCryptPasswordEncoder` (work factor ≥ 10).



2. Distinción Estricta Authn vs Authz:

Autenticación produce 401; autorización produce 403. Mantener ambos flujos desacoplados.



3. Autorización Granular:

Asegurar controladores con permisos atómicos y mapear roles a conjuntos de permisos en persistencia.



4. Centralización en SecurityFilterChain:

Configurar reglas de seguridad de forma explícita y declarativa en el bean principal.